



UDEM

UNIVERSIDAD DE MONTERREY

Práctica 2

Microprocesadores

M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo García Mat: 612348

Nombre: Ricardo Garza Benítez Mat: 645972

"Damos nuestra palabra de que hemos hecho esta actividad con integridad académica"

Monterrey, N. L., a 4 de Junio del 2026

Introducción

La matriz de LEDs 8×8 es un dispositivo de visualización formado por 64 diodos organizados en filas y columnas, cuyo encendido selectivo se logra activando simultáneamente una línea de fila y una de columna. En esta práctica se utilizó el PIC16F887 operando con un cristal de cuarzo de 16 MHz para controlar dicho display, con el objetivo de familiarizarse con las diferentes funciones del microcontrolador al momento de manejar periféricos de salida.

Para lograrlo se plantearon dos actividades. La primera consistió en desplegar un patrón estático en forma de X sobre la matriz, lo cual requirió definir los bytes de patrón para cada fila del display. La segunda actividad implicó generar una secuencia animada que muestra las letras R, I, E y U, correspondientes a las iniciales de los integrantes del equipo, aplicando multiplexación dinámica por software para que cada carácter se perciba de forma continua.

Ambas actividades se desarrollaron en lenguaje C dentro del entorno MPLAB X IDE con el compilador XC8, verificándose primero en simulación con Proteus antes de trasladarse al circuito físico en protoboard. La organización de los puertos del PIC16F887 en registros de ocho bits resulta especialmente adecuada para este tipo de periférico, ya que permite escribir el patrón completo de columnas en una sola operación sobre el Puerto D y seleccionar la fila activa mediante el Puerto B.

Marco Teórico

Matriz de LEDs 8×8 y Multiplexación Temporal

Una matriz de LEDs 8×8 agrupa 64 diodos en una cuadrícula donde cada LED se encuentra en la intersección de una línea de fila y una de columna. Para encender un LED determinado basta con activar su fila y su columna simultáneamente; sin embargo, dado que varias columnas comparten la misma fila, encender múltiples LEDs en distintas filas al mismo tiempo requeriría tantas líneas de control como LEDs independientes. La

solución a esta limitación es la multiplexación temporal: el microcontrolador activa una sola fila a la vez, coloca en las líneas de columna el patrón de esa fila, espera un breve retardo y avanza a la siguiente. Si este ciclo se repite con suficiente frecuencia, la persistencia retiniana integra las imágenes parciales y el ojo percibe el patrón completo sin parpadeo [1].

Para una matriz de 8 filas con un retardo de 2 ms por fila, la frecuencia de refresco resulta en aproximadamente 62.5 Hz, valor suficiente para eliminar el parpadeo perceptible. En la implementación de esta práctica, el Puerto B seleccionó la fila activa mediante un desplazamiento de bit, mientras que el Puerto D recibió el complemento del byte de patrón, dado que las columnas del display físico tienen lógica negada: un "0" en la línea de columna enciende el LED correspondiente. Esta inversión fue precisamente uno de los ajustes que el equipo debió identificar y corregir durante las pruebas físicas [2].

PIC16F887: Puertos de E/S y Configuración

El PIC16F887 es un microcontrolador de arquitectura RISC de 8 bits fabricado por Microchip Technology. Cuenta con 35 pines de entrada/salida digitales distribuidos en cinco puertos, cada uno controlado por su registro TRISx para dirección y PORTx para datos. Escribir cero en un bit del registro TRISx configura ese pin como salida; escribir uno lo configura como entrada. En la práctica, los registros TRISB y TRISD se inicializaron en 0x00 para habilitar ambos puertos completos como salidas digitales [2].

El oscilador del PIC16F887 se configuró en modo HS (High Speed) mediante los bits de configuración, conectando un cristal de cuarzo de 16 MHz entre los pines OSC1/CLKIN y OSC2/CLKOUT. Cada ciclo de instrucción consume cuatro ciclos de reloj, resultando en una frecuencia efectiva de ejecución de 4 MIPS. Este parámetro es fundamental para que las funciones de retardo del compilador XC8 sean precisas, ya que la macro `_XTAL_FREQ` debe declararse con el valor de 16,000,000 para calibrar correctamente los retardos de multiplexación [2].

Actividad 1 — Patrón Estático: X

Para desplegar una figura estática en la matriz se define un arreglo de ocho bytes donde cada byte representa el patrón de columnas de una fila. En el caso de una X, cada fila activa exactamente dos LEDs ubicados en las diagonales principal y secundaria: fila 0 en columnas 0 y 7, fila 1 en columnas 1 y 6, convergiendo hacia el centro. Estos valores se traducen directamente a hexadecimal y se almacenan en el arreglo de datos que el código escribe sobre el Puerto D en cada ciclo de multiplexación, manteniendo el patrón visible de forma continua dentro del bucle principal [1].

Actividad 2 — Secuencia Animada: R, I, E, U

Para mostrar caracteres en la matriz se utiliza una fuente de mapa de bits donde cada letra se codifica como ocho bytes, uno por columna. El arreglo bidimensional letras[4][8] almacena las definiciones de R, I, E y U, y el código itera sobre ellas mostrando cada carácter durante 800 ms. La duración de cada letra se controla calculando cuántas repeticiones completas del ciclo de ocho filas (a 2 ms cada una) suman ese tiempo, resultado en 50 repeticiones por letra. Este esquema de tres bucles anidados, uno por letra, uno de repetición y uno por fila, es característico de los sistemas de multiplexación temporal por software en microcontroladores de 8 bits [2].

Simulación

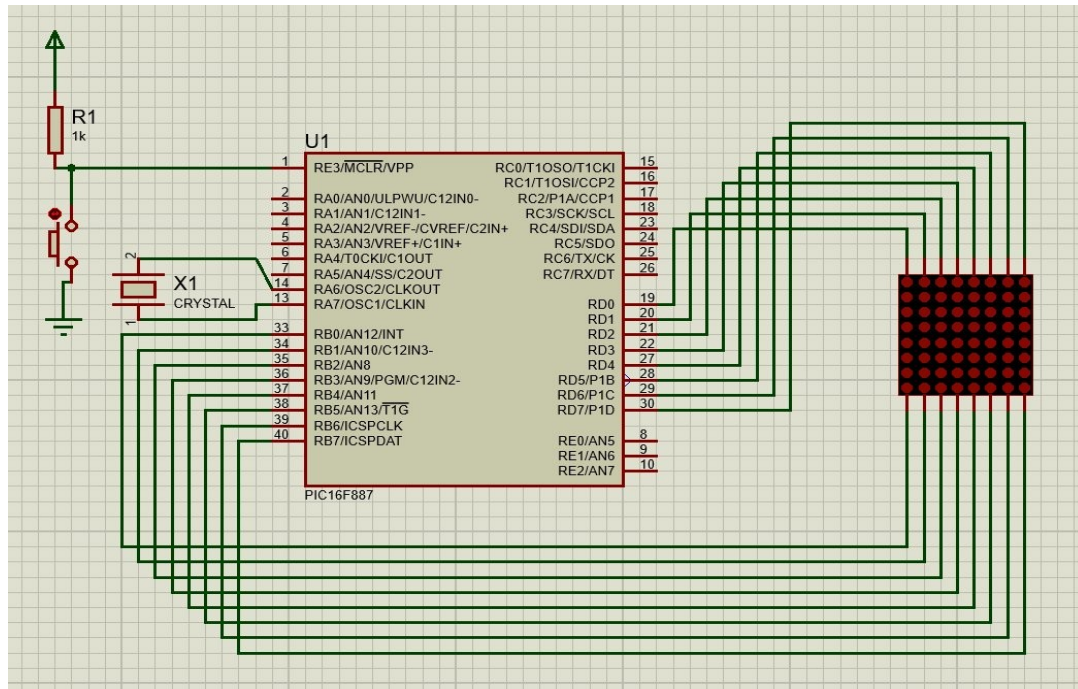


Foto del Circuito en Físico

[Foto del circuito físico en protoboard]

Resultados

Durante la Actividad 1, el patrón en forma de X se desplegó correctamente sobre la matriz con una imagen visualmente continua y sin parpadeo. El principal obstáculo fue la inversión entre filas y columnas respecto a lo esperado en simulación: al trasladar el circuito al protoboard físico, los patrones producían la figura rotada o reflejada respecto al display real. Fue necesario revisar la correspondencia de pines del componente físico y ajustar el mapeo de los bytes de patrón hasta obtener la orientación correcta. Una vez resuelto, la X se proyectó de forma nítida y el comportamiento coincidió al 100% con lo esperado.

En la Actividad 2 la secuencia R, I, E, U se ejecutó de manera fluida y legible. Cada letra permaneció visible los 800 ms definidos, y la transición entre caracteres fue inmediata sin efectos de ghosting. El mismo problema de inversión de filas y columnas se repitió en esta actividad, por lo que los patrones almacenados en el arreglo requirieron el mismo

ajuste de orientación. Una vez corregido, todas las letras se mostraron con claridad y la secuencia recorrió los cuatro caracteres de forma continua e indefinida. Ambas actividades concluyeron exitosamente.

En términos generales, la práctica mostró que el PIC16F887 tiene capacidad suficiente para gestionar una matriz 8×8 completamente por software. El cristal de 16 MHz garantizó la precisión de los retardos de multiplexación, y la correspondencia directa entre los registros de ocho bits del microcontrolador y las ocho líneas del display simplificó considerablemente el código de control.

Conclusiones Individuales

Ricardo Garza:

Esta práctica me resultó muy interesante porque fue la primera vez que vi de forma directa cómo la multiplexación temporal pasa de ser un concepto teórico a algo físicamente visible. Cuando armé el circuito y las filas y columnas aparecieron invertidas respecto a la simulación, entendí que el problema no era del código sino de la correspondencia entre el modelo de Proteus y el componente real, algo que solo se resuelve revisando la hoja de datos y rastreando pin por pin. Resolver eso manualmente en el protoboard me dejó claro que conocer el hardware real es igual de importante que dominar el software que lo controla, y que en sistemas embebidos ambas cosas tienen que ir de la mano desde el inicio del diseño.

Eugenio Romo:

En esta práctica pude entender de manera más concreta cómo el concepto de multiplexación temporal funciona aprovechando dos fenómenos físicos: la velocidad del microcontrolador y la persistencia de la visión humana. Lo que más me llamó la atención fue que el mismo mecanismo de control resuelve dos problemas distintos, un patrón estático y una secuencia animada, simplemente cambiando los datos del arreglo. Esa separación entre lógica de control y datos me parece un principio de diseño valioso que se puede aplicar a sistemas más complejos.

Referencias

[1] Merino, L. A. (2007). *Microcontroladores PIC: Diseño práctico de aplicaciones* (2.^a ed.). McGraw-Hill.

[2] Microchip Technology. (2009). *PIC16F887 Data Sheet*.
<https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>